Ambition Roadmap and Decision Register

Nov 4, 2026 · @dinky

Summary

Ambition now runs on AWS from week 1 and keeps its published 12-week plan and January 26, 2027 demo. It adopts the chapters, disciplines and architecture of *Building the Agentic Enterprise* and replaces the book's Google Cloud stack with AWS. The book follows a fictional firm, ACME Research, as its Macro Research & Education Platform (MREP) grows from one agent into a governed multi-agent platform.

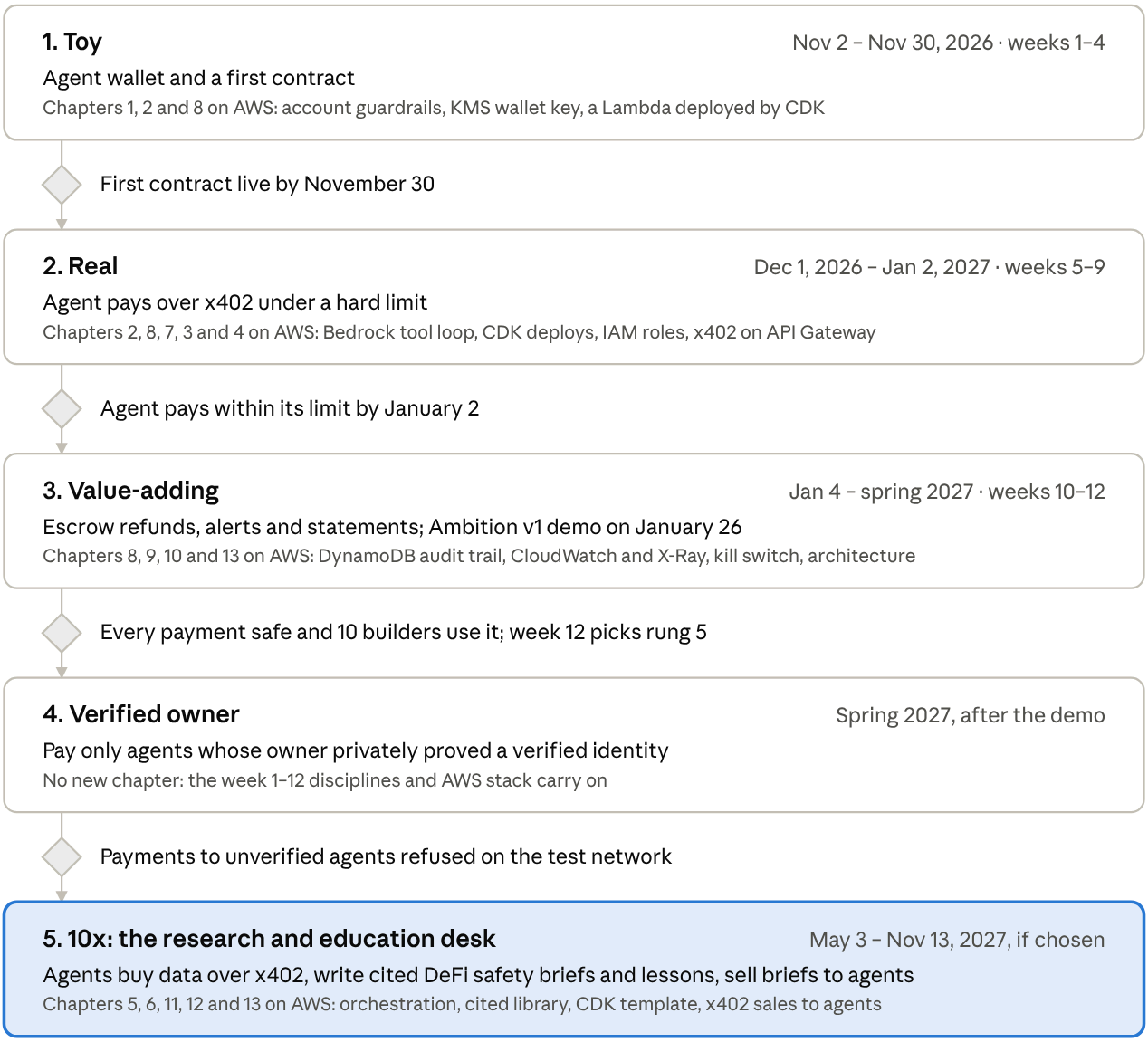
AWS is the cloud taught in the MSAIE's Cloud Applications and Architectures specialization, so the capstone practices what the degree teaches. Python stays the only language, because Boto3, AWS's Python toolkit, drives every AWS service. Base Sepolia still settles every payment, and GitHub Actions still tests and deploys.

Nine chapters land inside weeks 1 to 12 on a serverless AWS stack, meaning AWS runs the servers and Ambition supplies only code. The agent runs on Lambda, its wallet key is created and used inside AWS KMS so it never sits in a file, and every payment leaves an audit record in DynamoDB. The other four chapters describe a DeFi research and education desk after the demo, which becomes rung 5 only if the week-12 evidence chooses it.

Two money rules now run side by side, because the blockchain spends test money while AWS bills real dollars. Agents still spend only play money, and a separate AWS budget alarm and spending block caps the cloud bill (D-21).

Roadmap at a glance

Thirteen chapters on AWS: nine ride the published ladder, four build the desk



Ambition ladder · 5 rungs, 4 gates, 13 book chapters

Rungs 1 to 3 keep the published 12 weeks and their dates; the book's chapters add checks, and AWS adds the platform they run on. Rung 5 becomes the desk only if week 12 chooses it.

Weeks 1 to 12: the book as the operating discipline

Nine chapters change how the published weeks are built, not what they build or when. Each row names the week it lands in, what it adds on AWS and a check anyone can run, so the demo proves a working system rather than a prototype that only runs on one laptop.

Book chapter	Lands in	What it adds to the published week, on AWS	Done when
1. Why agentic systems fail	Weeks 1–2 (Nov 2–14)	Opens this Decision Register and runs each module through the book's Trinity of Value test before it is built, which guards against the PoC Trap (a demo that never reaches real use). The AWS account starts with its guardrails: a protected root login, a day-to-day admin login and a budget alarm	The register is in the repo, every later week has a one-line Done-When check, and the account passes its guardrail checklist
2. The agentic AI foundation	Weeks 1–6 (Nov 2–Dec 12)	The agent's reason-act-observe loop runs in Lambda and calls Claude through Bedrock's Converse API, which supports tool use: read a balance, pay, call the x402 service, then check the result	In week 6 the agent buys from the x402 service unaided, and CloudWatch shows every tool call with its inputs and result

Book chapter	Lands in	What it adds to the published week, on AWS	Done when
8. Building and deploying agentic workflows	Weeks 1–12 (Nov 2–Jan 23)	The book's Terraform and Cloud Run deployment becomes an AWS CDK stack in Python, deployed by GitHub Actions. Each module ships with its Done-When written as a test, and every run writes a structured log and stops at its budgets	Every merge to main deploys the stack, and the week-9 audit post is generated from CloudWatch Logs Insights queries
7. Security, identity and guardrails	Weeks 7–8 (Dec 14–26)	The book's zero-trust rules become IAM: one least-privilege role per Lambda, a KMS key only the agent's role may sign with, and GitHub deploying through OIDC. The week-8 checklist maps each OWASP agentic risk to a control	Each of the ten OWASP risks has a named control or an accepted-risk entry, and no long-lived AWS access key exists
3. Single-agent and multi-agent systems	Week 9 (Dec 28–Jan 2)	Agent version 1 stays one Lambda, and the register records the measurable trigger that would justify a second	Register entry D-06 names the trigger

Book chapter	Lands in	What it adds to the published week, on AWS	Done when
4. Core architectural patterns	Weeks 9–11 (Dec 28–Jan 16)	Control is split from execution: the model proposes a payment, a guard function checks it against the limit and allow-list, and only then may KMS sign. A reconciliation step after each payment is the book's reflection pattern	A test proves a fooled prompt cannot reach a signature without passing the guard
9. State, memory and decision persistence	Weeks 10–11 (Jan 4–16)	Statements become the decision audit trail in DynamoDB, keyed by wallet address, and escrow plus approval above the limit become the human approval gates	Any payment traces from its statement line to its request, evidence, version, approver and transaction hash
10. AgentOps and EvalOps	Weeks 11–12 (Jan 11–23)	CloudWatch and X-Ray carry the observability signals. The kill switch revokes the spending permission, disables the agent's schedule and denies its IAM role, and a GitHub Actions evaluation gate blocks any merge that fails a catastrophic test	Catastrophic tests pass 100% on every pull request, and the kill switch stops a test agent within a minute

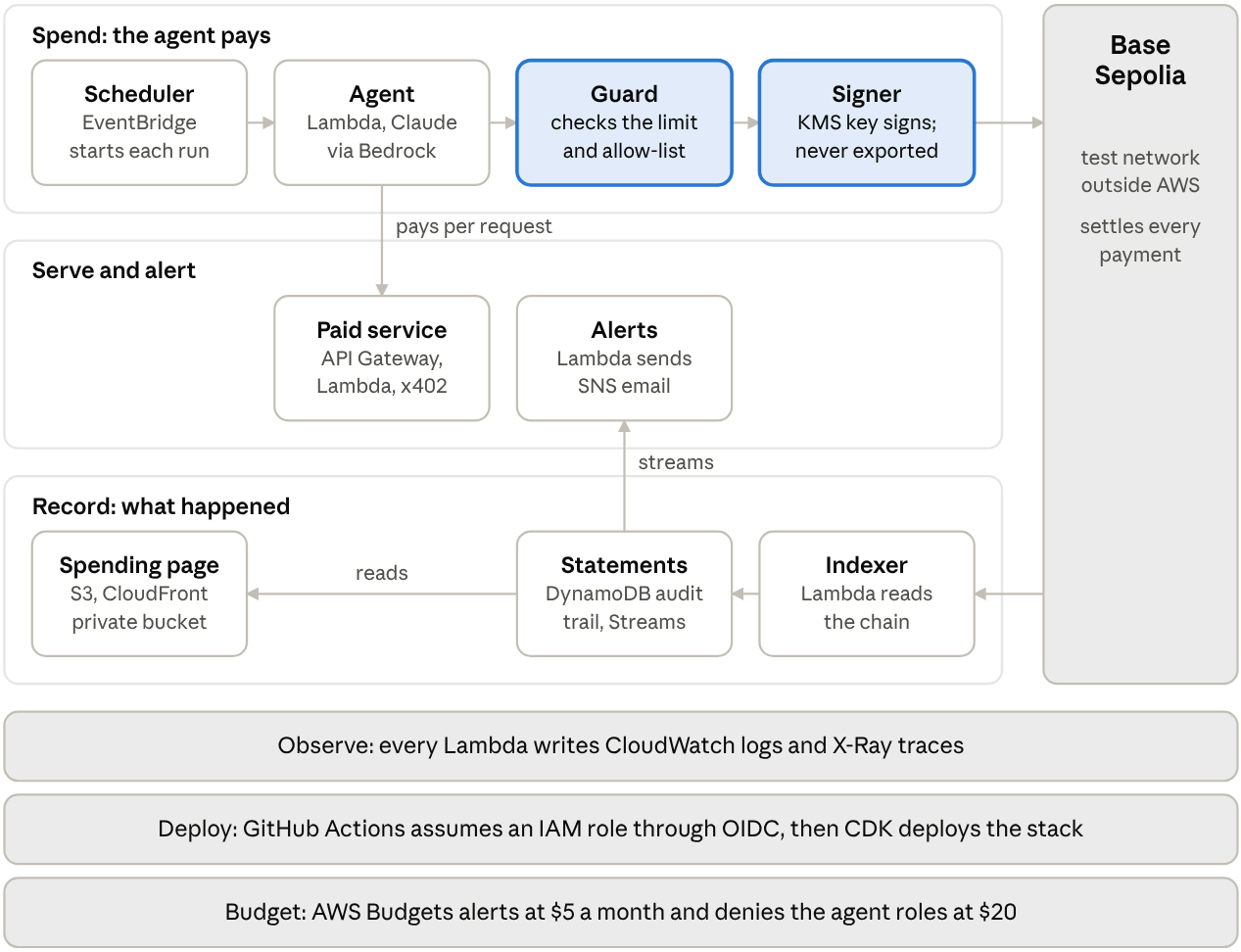
Book chapter	Lands in	What it adds to the published week, on AWS	Done when
13. The reference architecture	Week 12 (Jan 18–23)	The capstone write-up includes the AWS reference architecture, a Well-Architected review and this register, the package a reviewer asks for before backing more work	The January 26 demo shows the architecture, the review and the register

Rows run in the order their weeks start, so chapter 8 comes third and chapter 7 comes before chapter 3. The safety rules from *Building AI Agents for Finance* already in the reading notes carry on unchanged.

The AWS architecture

Every payment passes a guard in code and a signature inside KMS before it reaches the chain. Each job Ambition does gets one managed AWS service, the cloud-native method the curriculum teaches, and no part runs on a server Ambition must patch.

Every payment passes the guard and the KMS signer before it reaches the chain



AWS reference architecture · 3 paths, 9 parts

The agent can propose any payment, but only the guard can pass one to the signer, and only the signer holds the key.

The design answers each of AWS's six Well-Architected pillars with a named control, and the week-12 review checks them again.

Pillar	Ambition's answer
Operational excellence	Only GitHub Actions deploys, through the CDK stack; every run writes a structured log, and the kill switch is rehearsed before the demo

Pillar	Ambition's answer
Security	One least-privilege role per Lambda, a wallet key that signs only inside KMS, OIDC instead of stored keys, and a protected root login
Reliability	Lambda and DynamoDB run across several Availability Zones; reads retry with backoff, while payments carry idempotency keys and never retry blindly
Performance efficiency	Each function gets its own memory size, and DynamoDB reads by wallet address, its fastest kind of lookup
Cost optimization	Lambda and DynamoDB stay inside their always-free allowances, no NAT gateway runs, and AWS Budgets alerts and then blocks (D-21)
Sustainability	Nothing runs while idle, and DynamoDB TTL clears stale records

The AWS build, week by week

AWS joins in week 1 with a small foundation, then adds one service at a time as each published module needs it. Each week's AWS work fits inside its 8 hours. If the week-1 foundation runs past Saturday, the post ships on the local script and the Lambda follows in week 2, because a gate moves a date, never the bar.

Week	Published build	What runs on AWS
1 (Nov 2–7)	An agent wallet on Base Sepolia and a script that reads its transactions	Account guardrails: a protected root login, a day-to-day admin login and a budget alarm. The wallet key is created in KMS, and the read script runs as a Lambda that CDK deploys from GitHub Actions through OIDC

Week	Published build	What runs on AWS
2 (Nov 9–14)	A notebook that turns the agent's activity into a statement	An indexer Lambda writes the agent's transactions to DynamoDB, and the notebook reads them with Boto3
3 (Nov 16–21)	First smart contract, written in Remix	No new service, because the contract lives on the blockchain; the CDK stack gains its first tests
4 (Nov 23–28)	The phone book deployed to Base Sepolia	A separate KMS deployer key, usable only by the GitHub deploy role, signs the deployment, so no deploy key sits on a laptop
5 (Nov 30–Dec 5)	Test USDC and a script that pays another wallet	The pay Lambda signs with KMS, and EventBridge Scheduler starts each run
6 (Dec 7–12)	A paid x402 service and an agent that pays it	The service runs on API Gateway and Lambda, and the agent's loop calls Claude through Bedrock
7 (Dec 14–19)	A daily spending limit on the blockchain	The limit stays in the contract, and a guard Lambda checks it and a DynamoDB allow-list before KMS signs
8 (Dec 21–26)	Tests and the security checklist	An IAM review, the OWASP mapping and a first Well-Architected review
9 (Dec 28–Jan 2)	Agent version 1 buys data and logs every payment	X-Ray traces every run, and the audit post comes from CloudWatch Logs Insights
10 (Jan 4–9)	Money-back escrow	The contract enforces the deadline, and a one-time EventBridge Scheduler schedule claims the refund once it passes
11 (Jan 11–16)	Spending alerts and a public spending page	DynamoDB Streams trigger an alert Lambda that emails through SNS, and CloudFront serves the page from a private S3 bucket

Week	Published build	What runs on AWS
12 (Jan 18–23)	The demo and the capstone write-up	The architecture diagram, the Well-Architected review and a cost report from AWS Cost Explorer

Readers never need an AWS account. Every reader challenge stays on the blockchain, and Ambition's library runs on a laptop, so the CDK stack is the reference deployment for builders who want it rather than a requirement (D-23).

After the demo: the research and education desk

If week 12 chooses it, the desk becomes rung 5 and runs from May 3 to November 13, 2027, one chapter at a time. It starts after the verified-owner gate, because the ladder lets each rung pass its gate before the next begins, and a missed gate moves these dates rather than lowering the bar.

The desk is Ambition's first serious user. Its agents get money only through Ambition's wallet, limit, escrow and statements, so every week of research also tests the toolkit under real load.

Book chapter	Lands in	What Ambition builds on AWS	Done when (starting targets)
5. Multi-agent orchestration	Desk 1 (May 3–Jun 12)	One research agent buys public data over x402 and writes a weekly DeFi safety brief and a beginner lesson. An analyst, an educator and an editor that supervises the others join one at a time, each only after evaluation shows a gain. Strands Agents and Bedrock AgentCore compete for the orchestration layer (D-24)	Four weekly briefs in a row, every claim cited and every run under budget

Book chapter	Lands in	What Ambition builds on AWS	Done when (starting targets)
6. Context, memory and knowledge	Desk 2 (Jun 14–Jul 24)	The context moat: a cited library in S3 of protocol docs, contract audits, exploit post-mortems and public sanctions lists, which name crypto wallet addresses, searched by meaning and by keyword. The search index is chosen by cost	A 50-question test set answered with sources, and "insufficient evidence" returned whenever the library lacks the answer
11. Platform engineering	Desk 3 (Jul 26–Sep 4)	A paved road: a CDK template that deploys a wallet, limit, statements, logs and evaluations into a reader's own AWS account, plus a guide for adding a desk agent. Runs longer than Lambda's 15-minute limit move to Fargate	One reader, not the maintainer, deploys an agent from the template that passes the evaluation suite
12. Operating models and business transformation	Desk 4 (Sep 6–Oct 16)	Service-as-Software: other builders' agents pay per brief over x402 through API Gateway on the test network, which is business-to-agent commerce. Each workflow gets a RACI saying who is responsible, accountable, consulted and informed	Ten outside agents have paid for a brief, every workflow has a RACI, and an evidence review decides whether to form the company

Book chapter	Lands in	What Ambition builds on AWS	Done when (starting targets)
13. The reference architecture, final	Oct 18–Nov 13	The AWS architecture, Well-Architected review and register updated for the final capstone submission	Submitted ahead of the December 2027 graduation; the capstone due date is still an open question

The desk researches risk and safety, never prices. It explains what could go wrong in a protocol or a counterparty and teaches beginners how to check, but it makes no price calls, trade signals or personal recommendations. That keeps it on the research-and-education side of the line, keeps the plan's rule that nothing here is financial advice, and plays to Ambition's mission.

The week-12 decision gets a fifth row

The desk competes on evidence like every other path, so it enters the plan as one more row of the week-12 table. The row is proposed (D-02) and goes into the build-in-public plan only once you accept it.

If at week 12…	Then
Readers keep completing the lesson challenges and builders pay the x402 service from their own code	Choose the research and education desk as rung 5, because it serves both signals at once: people who want to learn and builders who will pay agents for data

How the books and the curriculum are used

Ambition adopts the book's chapters, disciplines and architecture, and swaps each Google Cloud piece for its AWS counterpart. The swap follows the cloud-native method the curriculum teaches: list each job the app must do, then give each job one managed service.

The book on Google Cloud	Ambition on AWS	Why this one
Agent Development Kit (ADK)	Bedrock's Converse API through Boto3 for one agent; Strands Agents or Bedrock AgentCore scored at the desk (D-24)	A plain loop keeps every reason-act-observe step visible while there is one agent
Gemini models	Claude on Amazon Bedrock, with pinned model IDs (D-13)	The Lambda's IAM role authorizes each call, so there is no model API key to leak
Cloud Run	AWS Lambda, with Fargate for runs over 15 minutes (D-18)	Lambda bills per request and leaves server patching to AWS
Workload identity	IAM roles, with GitHub OIDC for deployments (D-20)	Short-lived credentials replace stored keys
VPC Service Controls and perimeter	Least-privilege IAM now; a VPC across two Availability Zones once a private resource exists (D-22)	Nothing private needs a network yet, and a NAT gateway bills every hour
Data residency on Cloud Run	One region, us-east-1 (D-22)	Every record stays in one region
ADK session and memory services	DynamoDB, keyed by wallet address	Fast reads by key, flexible records and Streams for alerts
Structured logging and tracing	CloudWatch Logs and X-Ray	Both are built into Lambda
Terraform	AWS CDK in Python (D-25)	Infrastructure lives in the same language as the code
Cloud Build	GitHub Actions	It is already Ambition's CI
Gemini Enterprise adoption layer	Not needed	Ambition's users are builders, not employees in an enterprise rollout

One piece has no counterpart in the book: the wallet key. It is created inside AWS KMS as a secp256k1 key, the type Ethereum and Base use, and it signs there, so the private key never leaves AWS's hardware (D-19).

Neither the books nor the course materials go into Git. This roadmap summarizes *Building the Agentic Enterprise* in its own words and cites it by chapter, the same rule the reading notes set for the other book. The Quantic specialization summary stays private too, because its copyright and the Student Code of Conduct bar sharing it, so every AWS fact here cites AWS's public documentation (D-15).

Starter Decision Register

Twenty-five decisions make up the register: seven accepted and eighteen proposed. It is Ambition's version of the MREP Decision Register from chapter 13, the record of every decision the design rests on. D-08, which kept Ambition off any cloud platform, was withdrawn on November 4, 2026, when AWS took its place (D-17), and its number is not reused. Change a status in its dropdown, and add each new decision as a row with the next ID.

ID	Decision	Status	Why	Revisit when	Chapter
D-01	The 12-week plan, its ladder and the January 26 demo stay as published; the book and AWS shape how they are built	Proposed	Readers already follow week 1, and a missed gate moves a date, never the bar	Not before the demo	1
D-02	The research and education desk is a week-12 option for rung 5, not a commitment	Proposed	Evidence, not enthusiasm, chooses rung 5	Week 12 (Jan 18–23)	1, 12
D-03	Agents spend only test money on Base Sepolia and never touch a real-money account	Accepted	Payments are final, and beginners copy what the build does	After an independent contract audit	7
D-04	The desk reads real-network data but spends only on the test network	Proposed	DeFi research needs real protocols' data, and reading moves no money	Desk 1 start	6

ID	Decision	Status	Why	Revisit when	Chapter
D-05	The desk covers risk, safety and lessons, with no price calls, trade signals or personal recommendations	Proposed	It stays research and education, matches the mission and keeps "not financial advice" true	Before any paid offer to people	12
D-06	One agent with tools carries weeks 1 to 12; a second joins only when evaluation shows a measurable gain	Accepted	More agents add cost and failure modes before they add value	When the evaluation suite shows one agent missing its Done-When because its context is too full	3
D-07	Control lives outside the model: the limit in the contract, the allow-list in the tool layer, approval above the limit	Accepted	A fooled model still cannot overspend	Only ever tightened	4, 7
D-09	Every claim in a brief or lesson carries a source, and "insufficient evidence" is a valid answer	Proposed	Readers can check the work, and a cited "don't know" beats a confident guess	Desk 2 test set	6
D-10	Statements are the decision audit trail: one record per payment binds request, evidence, version, guardrail decisions, approver and transaction hash	Accepted	Anyone can trace why money moved	Week 10	9
D-11	Catastrophic failures (overspending, a wrong recipient, a leaked key) must pass 100% in a CI evaluation gate	Proposed	The reading notes already demand 100% on these, and CI makes the check automatic	Week 11	10
D-12	The kill switch revokes the agent's spending permission, disables its EventBridge schedule and attaches an IAM deny to its role	Proposed	One action stops the spending, one stops the runs and one stops every AWS call, all within seconds	Week 11	10
D-13	Each release pins model IDs and hashes prompts, so a prompt change is a code change	Accepted	Any result traces to the exact version that produced it	Every release	8, 10

ID	Decision	Status	Why	Revisit when	Chapter
D-14	The week-8 checklist maps every OWASP agentic risk to a control or an accepted risk	Proposed	A standard list makes the security review harder to skip than a homemade one	Week 8	7
D-15	Book PDFs and course materials stay out of Git; docs summarize them in their own words and cite chapters, pages or public sources	Accepted	The repo is public and mirrored, and the books' copyright, the course's copyright and the Student Code of Conduct bar sharing them	Never	—
D-16	Briefs are sold to other agents over x402 on the test network before any real-money pricing	Proposed	It proves business-to-agent demand at no risk	Desk 4	12
D-17	AWS is Ambition's cloud platform from week 1, replacing the book's Google Cloud; Python, Base Sepolia and GitHub Actions keep their roles	Accepted	The MSAIE specialization teaches AWS, and Boto3 keeps every service in Python	Week-12 cost and Well-Architected review	2, 8
D-18	Serverless first: Lambda, API Gateway, DynamoDB, S3 and CloudFront, with no EC2 servers or bastion hosts	Proposed	Under the shared responsibility model AWS patches managed services, and idle functions cost nothing	When a run needs more than Lambda's 15 minutes, which moves it to Fargate	4, 8
D-19	The wallet key is created inside AWS KMS as a secp256k1 key and signs there; no private key sits in a file, variable, prompt or repo	Proposed	A key that never leaves KMS can't leak through the model, a log or a commit	If signing latency or cost blocks a use case	7
D-20	One least-privilege IAM role per Lambda, and GitHub Actions deploys through OIDC; no long-lived AWS access keys exist	Proposed	Short-lived credentials can't be stolen from a repo secret or a laptop	Week-8 IAM review	7

ID	Decision	Status	Why	Revisit when	Chapter
D-21	The AWS bill has its own limit: AWS Budgets alerts at $5 a month and applies an IAM deny to the agent roles at $20	Proposed	Cloud costs are real dollars even while the blockchain's money is not	Desk 1 start	10, 12
D-22	One region, us-east-1, and no VPC until a private resource exists; then two Availability Zones with public and private subnets	Proposed	Lambda and DynamoDB already span several Availability Zones, and a NAT gateway bills every hour it exists	Desk 2, if a private database is added	7
D-23	Ambition's library runs on a laptop with no cloud account; the CDK stack is the reference deployment, and reader challenges never need AWS	Proposed	Readers stay free and sign-up-free, and the toolkit stays portable	Desk 3 paved road	11
D-24	The single agent calls Bedrock's Converse API directly through Boto3; Strands Agents or Bedrock AgentCore joins at the desk only if it wins the nine-dimension scoring from *Building AI Agents for Finance*	Proposed	A plain loop shows every reason-act-observe step while there is one agent	Desk 1 start	2, 5
D-25	Infrastructure is code in AWS CDK (Python), tested like the rest of the code	Proposed	One language covers the app and its infrastructure, replacing the book's Terraform	If CDK blocks a needed service	8, 11
D-26	The escrow contract enforces deadlines, a one-time EventBridge Scheduler schedule claims refunds, and DynamoDB TTL only tidies old records	Proposed	AWS deletes expired TTL items "within a few days", not at the deadline	Week 10	9

Plain-language terms

Every term this roadmap adds is defined here, in the same plain style as the design guidelines' glossary.

Term	Plain meaning
API Gateway	An AWS service that gives code a web address and decides which function answers each request
Audit trail	A record that shows what an agent did, why, with which version and who approved it, so anyone can check it later
Availability Zone	One of several separate data centers inside an AWS region, so a local outage doesn't stop the app
AWS Budgets	AWS's spending alarms, which can also block chosen roles once the bill passes a set amount
AWS CDK	A toolkit for describing AWS resources in Python code, so the whole setup can be reviewed, tested and rebuilt
Base Sepolia	Coinbase's free practice blockchain, where all of Ambition's money is test money
Bedrock	AWS's service for calling AI models such as Claude, authorized by AWS roles instead of a separate API key
Boto3	AWS's Python library for calling AWS services from code
Business-to-agent commerce	Selling to other companies' AI agents rather than to people, here by charging per request over x402
CloudFront	AWS's content-delivery network, which serves a web page quickly while its storage bucket stays private
CloudWatch	AWS's logs and metrics service, where every Lambda writes what it did

Term	Plain meaning
Context moat	Knowledge a competitor can't easily copy, fed to the agents so their answers beat a general model's
Contract audit	An expert review of a smart contract's code that looks for ways money could be lost
Counterparty	Whoever is on the other side of a payment or deal, such as the wallet an agent pays
Decision Register	One table of every decision the design rests on, with its reason and when to revisit it
DeFi	Decentralized finance: lending, trading and payments run by smart contracts instead of a bank
Done-When	A check anyone can run that says a piece of work is finished, written before the work starts
DynamoDB	AWS's database that stores records by a key and needs no server to manage
Evaluation gate	An automatic test run that blocks a change from merging if the agent fails a critical test
EventBridge Scheduler	An AWS service that starts a function at a set time or on a repeating schedule
Exploit post-mortem	A write-up of how a protocol lost money to an attack and what fixed it
IAM role	A set of AWS permissions that code takes on for a short time, instead of holding a password or key
Kill switch	A fast way to stop an agent: here, cancelling its spending permission, pausing its schedule and denying its role
KMS	AWS Key Management Service, which keeps keys inside secure hardware and signs with them there
Lambda	AWS's way to run a function without managing a server; it runs on demand and bills per request
NAT gateway	A paid AWS router that lets private machines reach the internet, billed every hour it exists

Term	Plain meaning
OIDC	A standard that lets GitHub prove who it is to AWS and receive short-lived credentials, so no AWS key is stored
Orchestration	One supervising agent hands tasks to helper agents and checks their work
PoC Trap	The book's name for a proof of concept that impresses in a demo but never reaches real use
Protocol	A DeFi service built from smart contracts, such as a lending market
RACI	A chart naming who is Responsible, Accountable, Consulted and Informed for each task
Reason-act-observe loop	How an agent works: it decides what to do, uses one tool, looks at the result, then decides again
Region	An AWS geographic area, such as us-east-1 in northern Virginia, made of several Availability Zones
S3	AWS's file storage, organized in buckets
Sanctions list	A government list of people, companies and, in the US, crypto wallet addresses that may not be paid
secp256k1	The type of cryptographic key Ethereum and Base use to sign transactions
Serverless	Building with services AWS runs and patches, so Ambition supplies only code and settings
Service-as-Software	The book's term for selling the finished work an agent does, rather than the software that does it
Shared responsibility model	AWS secures the cloud itself, and the customer secures what it puts in the cloud
SNS	AWS's notification service, used here to email alerts
Time to live (TTL)	A DynamoDB setting that deletes old records automatically, within a few days of a chosen time

Term	Plain meaning
VPC	A private network inside AWS, split into smaller networks called subnets
Well-Architected Framework	AWS's six-pillar checklist for judging a cloud design
x402	Coinbase's standard for agents paying for things on the web, one request at a time
X-Ray	AWS's tracing service, which follows one request through every function it touches

Sources

- Jaroslav Pantsjoha, *Building the Agentic Enterprise* (Packt, September 2026), the MREP case study and the chapter structure mapped here
- Hanane Dupouy and Fayssal El Mofatiche, *Building AI Agents for Finance* (Packt, 2026), with Ambition's adopted rules in the reading notes
- Ambition, Build in Public: Mission and 12-Week Plan, the weeks, ladder, gates and week-12 decision this roadmap keeps
- OWASP GenAI Security Project, OWASP Top 10 for Agentic Applications for 2026 (December 9, 2025)
- US Treasury, OFAC FAQ 563, on how the sanctions list records digital currency addresses
- AWS, Well-Architected Framework, Sustainability Pillar, which lists the framework's six pillars
- AWS, Lambda quotas, on the 15-minute function timeout
- AWS, Converse API reference, on tool use through `toolConfig`
- Chainlink, AWS KMS Signer, on signing with an `ECC_SECG_P256K1` KMS key whose private key never leaves AWS's hardware
- GitHub, Configuring OpenID Connect in Amazon Web Services, on deploying without long-lived AWS secrets
- AWS, Configuring budget actions, on applying an IAM policy when a budget passes its threshold
- AWS, Pricing for NAT gateways, on hourly and per-gigabyte charges

- AWS, Using time to live (TTL) in DynamoDB, on deletion "within a few days" and TTL deletes in Streams
- AWS, Invoke a Lambda function on a schedule, on one-time and recurring EventBridge Scheduler schedules
- AWS, Restrict access to an Amazon S3 origin, on keeping the bucket private behind CloudFront
- AWS Builder Center, Everything you need to know to get started with AWS Free Tier (May 2026), on credits and always-free Lambda and DynamoDB allowances
- AWS Builder Center, The AWS agentic stack explained: Strands, AgentCore, MCP and A2A, on Strands Agents and AgentCore's October 2025 general availability